Predicting and Preventing 30-Day Hospital Readmissions

Full Name: Le Quang Thien Nguyen

Student Number: 300354247

Course: CSIS4495

Section Number: 002

Introduction:

First of all, as I shared with you a readmission within 30 days of discharge is one of the most closely watched numbers in healthcare. Medicare financially penalizes US hospitals whose 30-day rates exceed expected benchmarks, and Canadian health authorities, including those in British Columbia, report the same indicator through the Canadian Institute for Health Information. The cost is two fold like this, a second inpatient stay is expensive, and it usually signals that something in the first discharge, the medication list, the follow-up plan, the handover did not hold. Hospitals already hold the data that would help: diagnosis, comorbidity count, length of stay, prior admissions, discharge disposition. What most lack is a systematic way to use it. Deciding who needs intensified follow-up still comes down to a checklist or a clinician's judgment under time pressure. This project treats that gap as an analytics problem: turn the discharge record into an automatic risk screen that runs for every patient, every day. And it also would be a website as you told me that has multiple features for third parties, or hospitals can use it to track and pay attention too high-risk patients like those people who have diabetes....

How does it work?

The central question: which patient-level, clinical, and administrative factors are most strongly associated with 30-day readmission, and can a machine-learning classifier flag high-risk patients early enough, and clearly enough, for a discharge-planning team to intervene before the patient leaves?

1. Readmissions are often preventable. A call, a home-care referral, or a medication review works, when it reaches the right patient. The hard part is knowing who that is while they are still on the ward.
2. Follow-up capacity is finite. No hospital can intervene with every discharged patient, so any useful system has to rank them.
3. A bare probability is not actionable. Staff need to know why a patient was flagged and what to do about it.

Literature Review:

Model choice: Based on the UCI "Diabetes 130-US Hospitals" dataset consistently find that ensemble tree methods beat logistic regression on accuracy, while logistic regression remains valuable as an interpretable baseline to measure that gain against. Prior inpatient admissions and discharge medication count repeatedly out predict individual diagnosis codes.

Class imbalance: Readmitted patients are typically 10–20 percent of discharges. A model trained without correction learns to predict "not readmitted" every time: high accuracy, no usefulness. Class-weighting and synthetic oversampling SMOTE(Synthetic Minority Over-sampling Technique) are the established corrections.

Explainability: Clinical stakeholders report that a black-box score is difficult to act on. SHAP(Shapley Additive explanations) values are the standard way to attribute a prediction to specific inputs, both across a population and for one patient.

The gap: Coursework-level work usually stops at the score. It rarely connects output to a specific recommended intervention, estimates what that intervention avoids, or checks whether performance holds across patient subgroups rather than reporting one aggregate figure. My project targets that gap.

Assumptions and limitations: The dataset is diabetic inpatient encounters from 1999–2008, not a general all-cause BC population, so results demonstrate method rather than transferable rates; the data is public and de-identified, so no ethics review is required; and a rule-based intervention layer over the model's risk tiers stands in for clinical protocol, which is outside the scope of a one-semester course project.

Potential benefit: The finished project shows end to end how a hospital could convert discharge data it already holds into a low-cost, explainable screening step, and what that step is worth against the cost of preventable readmissions.

Proposed Research Project:

Research Design, Objectives, and Justification:

My project is a quantitative, predictive-modeling study in three layers: an exploratory layer that quantifies how each predictor relates to readmission, a modeling layer that builds an interpretable baseline and compares ensembles against it, and an application layer that turns output into an explainable, per-patient decision. I follow from **Literature Review** exploration alone cannot support a predictive claim, and a model without an application layer is exactly the black-box gap the literature identifies. Together they satisfy the assignment's requirement for a deliverable with a practical contribution.

Objectives:

1. Quantify the association between each candidate predictor and 30-day readmission.
2. Build a logistic regression baseline and compare it against random forest and XGBoost, correcting for class imbalance.

3. Apply SHAP analysis for global feature importance and per-patient explanations.
4. Test whether performance and calibration hold across demographic and diagnosis subgroups.
5. Translate output into three risk tiers, each mapped to a recommended intervention and an estimated cost avoided.
6. Deliver the result as a web application: a React front-end over a FastAPI machine-learning service.

Data, Sample, and Analysis Techniques:

Data source: After searching out, I found this dataset on the UCI "Diabetes 130-US Hospitals" dataset consistently find that ensemble tree methods beat logistic regression on accuracy, while logistic regression remains valuable as an interpretable baseline to measure that gain against. Prior inpatient admissions and discharge medication count repeatedly out predict individual diagnosis codes.

The UCI Machine Learning Repository's "Diabetes 130-US Hospitals for Years 1999–2008": over 100,000 de-identified inpatient encounters across 130 hospitals, roughly 50 features each, with the outcome recorded as readmitted within 30 days, readmitted later, or not readmitted. Every encounter carries a diabetes diagnosis and a stay of 1 to 14 days, so the cohort is narrower than the all-cause indicator described in Section 1.1: a limitation stated in the findings rather than a claim made for them. Using an established public dataset removes the collection risk a semester-long scraping effort would carry for a solo student. This is the link of the dataset: https://archive.ics.uci.edu/dataset/296/diabetes+130-us+hospitals+for+years+1999-2008

Sampling and splitting: The dataset records multiple encounters per patient, so a row-level random split would place the same patient in both training and test data and inflate measured performance. The split is therefore grouped by patient identifier as well as stratified on the outcome: across train, validation, and test so no patient appears in more than one partition and each preserves the true proportion of readmitted encounters.

Runtime inputs: Training is a one-time batch load. The application scores records arriving three ways: a replayed cohort drawn from the held-out test partition and presented as a day's discharge list, whose known outcomes let the tool show which flagged patients were in fact readmitted. A CSV upload returned as a ranked worklist, mirroring how an analyst would use it before any integration and a single-patient form for ad-hoc scoring. All three call the same endpoint.

Prediction timing: Discharge medication count and discharge disposition are known only once the discharge decision is made, so the intervention point is the moment the discharge

order is written, not admission. That is still early enough to arrange a follow-up call or referral, but it is a narrower claim than predicting readmission on arrival.

Analysis Technique:

Mostly, I will use Python language, combine with SQL and other technique...

1. **Data cleaning** — resolve missing and placeholder values (the dataset uses "?"), drop encounters with no recorded discharge disposition, and collapse high-cardinality diagnosis codes into clinical groups.
2. **Exploratory analysis** — readmission rate cross-tabulated against age band, diagnosis group, discharge disposition, admission type, prior admissions, medication count, and length of stay; computed in SQL, visualized in Python.
3. **Imbalance correction** — class-weighting as the primary method, with SMOTE evaluated as an alternative on validation performance.
4. **Modeling** — logistic regression as the interpretable baseline, against random forest and XGBoost, tuned by cross-validated grid search.
5. **Calibration** — because the cost model multiplies predicted probabilities by dollar figures, those probabilities must mean what they say. Calibration is assessed by Brier score and calibration curve, with Platt scaling or isotonic regression applied if the raw ensemble output proves miscalibrated.
6. **Explainability** — SHAP values for the best model, as a global importance summary and as per-patient attributions served through the application.
7. **Subgroup analysis** — precision, recall, AUC, and calibration recomputed by age band, recorded race, sex, and diagnosis group to expose uneven performance.
8. **Decision layer** — three tiers at probability thresholds read off the validation precision-recall curve, each mapped to one action: routine discharge instructions; a 48-hour follow-up call; or a home-care referral plus medication reconciliation.
9. **Cost-benefit estimation** — using a cited average cost of an avoidable readmission and an estimated cost per tier, both sourced during phase 2, net financial impact is projected at several candidate thresholds so the final threshold is chosen on evidence rather than by eye.

Technologies:

The deliverable is a single web application, a React front-end over a Python machine-learning service. It keeps the interface and the model in one system means the risk score,

the explanation behind it, and the recommended action appear on the same screen, which is the point the project is making.

Layer	Technology
Languages	Python 3.11 (modeling and back-end); TypeScript (front-end); SQL
Data	PostgreSQL — encounters fact table with dimension tables for demographics, diagnosis group, and discharge disposition; SQL views for cohort and subgroup queries
Modeling	pandas, scikit-learn, imbalanced-learn, XGBoost, SHAP; pipeline persisted as a versioned model artifact
Back-end	FastAPI — endpoints for cohort scoring, CSV upload, single-patient prediction with SHAP attribution, model metrics, and threshold/cost analysis; SQLAlchemy and Pydantic
Front-end	React with TypeScript (Vite), Recharts. Five screens: cohort intake, discharge queue, patient detail with explanation and action, model and fairness report, threshold and cost analysis
Tooling	Git and GitHub; GitHub Projects Kanban; pytest and Vitest; Docker Compose for local PostgreSQL; containerized deployment to a free-tier host

The PostgreSQL schema exposes BI-ready views, so an operational deployment could connect Power BI or Tableau to the same data. Building those dashboards is out of scope: the web application covers the same reporting, and splitting it across three tools would fragment the decision workflow this project argues for. But for some reasons, I will figure out which technique that I use during doing the project, and I will update it to you more.

Project Planning and Timline:

As I'm doing solo project, and I am responsible for every phase. Work runs in six phases of two to three weeks, each ending in a checked-in deliverable.

Phase	Weeks	Deliverable
1. Setup and literature review	1–2	Proposal finalized; dataset validated; PostgreSQL schema designed; repository initialized with API and front-end scaffolds
2. Data cleaning and EDA	3–4	Cleaned dataset loaded to PostgreSQL; SQL exploratory queries; cost figures sourced and cited; first-pass charts
3. Baseline modeling	5–6	Logistic regression baseline trained and evaluated; imbalance strategy chosen; Midterm Report submitted
4. Ensemble modeling and tuning	7–9	Random forest and XGBoost trained, tuned, and compared; calibration and subgroup analysis completed
5. Explainability and API	10–11	SHAP integration; FastAPI service exposing scoring, upload, explanation, and metrics endpoints; risk-tier and cost logic
6. Web application and writeup	12–13	React application built and deployed; Final Report and presentation prepared

All responsibilities: data engineering, modeling, API, front-end, and reporting are carried out by the sole team member. A GitHub Projects Kanban board (Backlog / In Progress / Review / Done) is maintained weekly and serves as the project management chart referenced here. And the timeline above can be adjusted, if any new updates that won't be in the timeline, I will follow up and update it to you.

Project Contract:

My name is Le Quang Thien Nguyen, student id.300354247, as solo member, agree to the scope, methodology, and timeline set out in this proposal for CSIS 4495. I agree to submit progress reports individually as required, to maintain the work log with an entry for each day work is performed, and to complete every milestone by its target week — notifying the instructor promptly if a milestone is at risk.

Signature: _____Thien_____ Date: _____28/09/2026_____

AI Use:

AI Tool Name	Version	Specific feature for which the AI tool was used	Value Addition What value did you add over and above what AI did for you?
Claude	Free	Plain-language summary; scope risk flagged in the three-dashboard deliverable	Supplied the draft proposal. Judged the summary against the actual document and accepted the scope concern as real
Claude	Free	"is there anyway to make the proposal cleaner and easy to understand, than the current way. keeping the overall structure. also switching to the web app"	Set the constraint that the seven-section structure was not to change
Claude	Free	Identify gaps in the proposal?"	Knowing the expected limitations of the product

Work Date and Hours Log:

Student name: Le Quang Thien Nguyen

Date	Hours	Work done
2026-09-17	1.5	Reviewed proposal draft; evaluated Streamlit/Power BI/Tableau stack against a single web application; decided on React + FastAPI + PostgreSQL

Date	Hours	Work done
2026-09-19	1	Produced UI mockups of five application screens: cohort intake, discharge queue, patient detail with SHAP explanation, model and fairness report, threshold and cost-benefit
2026-09-19	1	Rewrote proposal for clarity; updated, and the timeline for the revised stack
2026-09-19	0.5	Specified runtime inputs (replayed test cohort, CSV upload, single-patient form); documented the prediction-timing constraint
2026-09-22	1	Methodology review; added patient-level grouped splitting, probability calibration, race in the subgroup analysis, cohort and ethics caveats, and a contingency plan
2026-09-24	0.5	Condensed proposal to 5 pages against the 6-page limit
Total	5.5	

Closing and References:

Strack, B., DeShazo, J. P., Gennings, C., Olmo, J. L., Ventura, S., Cios, K. J., & Clore, J. N. (2014). Impact of HbA1c Measurement on Hospital Readmission Rates: Analysis of 70,000 Clinical Database Patient Records. BioMed Research International.

Centers for Medicare & Medicaid Services. Hospital Readmissions Reduction Program (HRRP). [Update with current URL and access date.]

Canadian Institute for Health Information. All-Cause Readmission to Acute Care indicator. [Update with current URL and access date.]

Lundberg, S. M., & Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. Advances in Neural Information Processing Systems.

Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. Journal of Artificial Intelligence Research.